Lecture 08
Deep Learning

Ph. D. Juan David Martínez Vargas
Ph. D. Raul Andrés Castañeda

Escuela de Ciencias Aplicadas e Ingeniería

Vigilada Mineducación

Agenda

- **Data Augmentation**
- **Transfer Learning**

Data Augmentation

Los modelos DL dependen fuertemente de la calidad y diversidad de los datos con los que se entrenan. Sin embargo, recolectar grandes conjuntos de datos suele ser costoso y demandar tiempo.

El **data augmentation** permite ampliar artificialmente el conjunto de entrenamiento generando nuevas versiones de los datos existentes.

- Incrementar el tamaño del dataset.
- Reducir el overfitting.
- Mejorar la capacidad de generalización del modelo.

- **Imágenes**
- **Audio**
- **Texto**
- **Series temporales**
- **Datos tabulares**

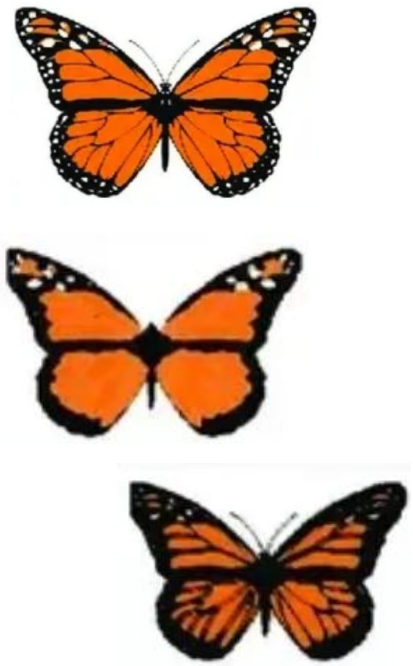
Técnicas de Data Augmentation

Image Augmentation→ Técnica utilizada en tareas de visión por computador para aumentar artificialmente el conjunto de entrenamiento mediante transformaciones aplicadas a las imágenes.

Transformaciones geométricas

Transformaciones fotométricas

Uso de Filtros



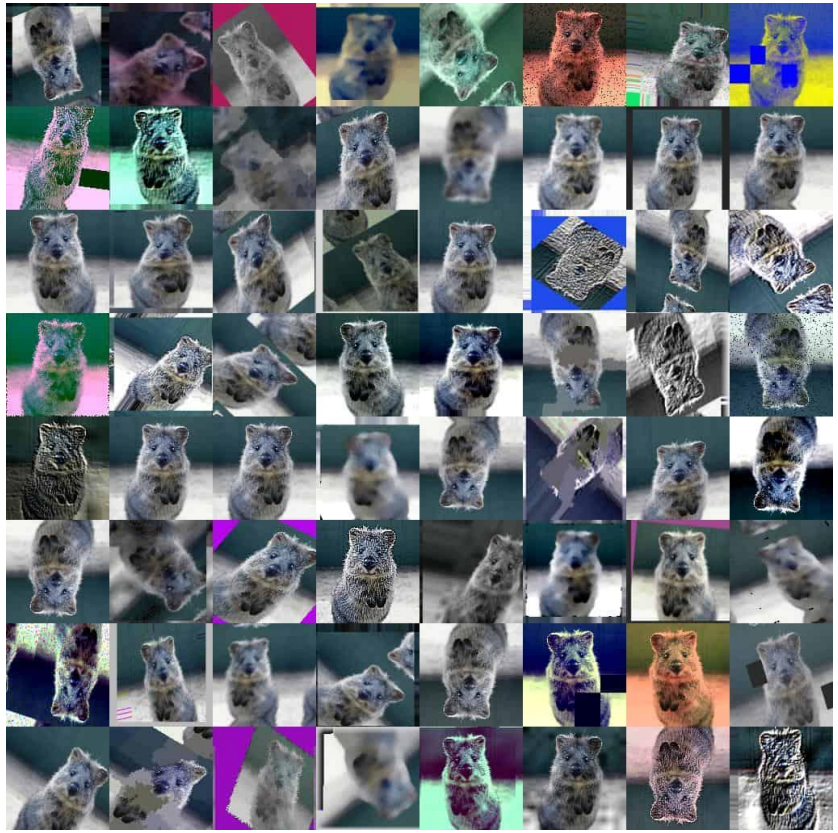
Mezclar imágenes

Borrar información

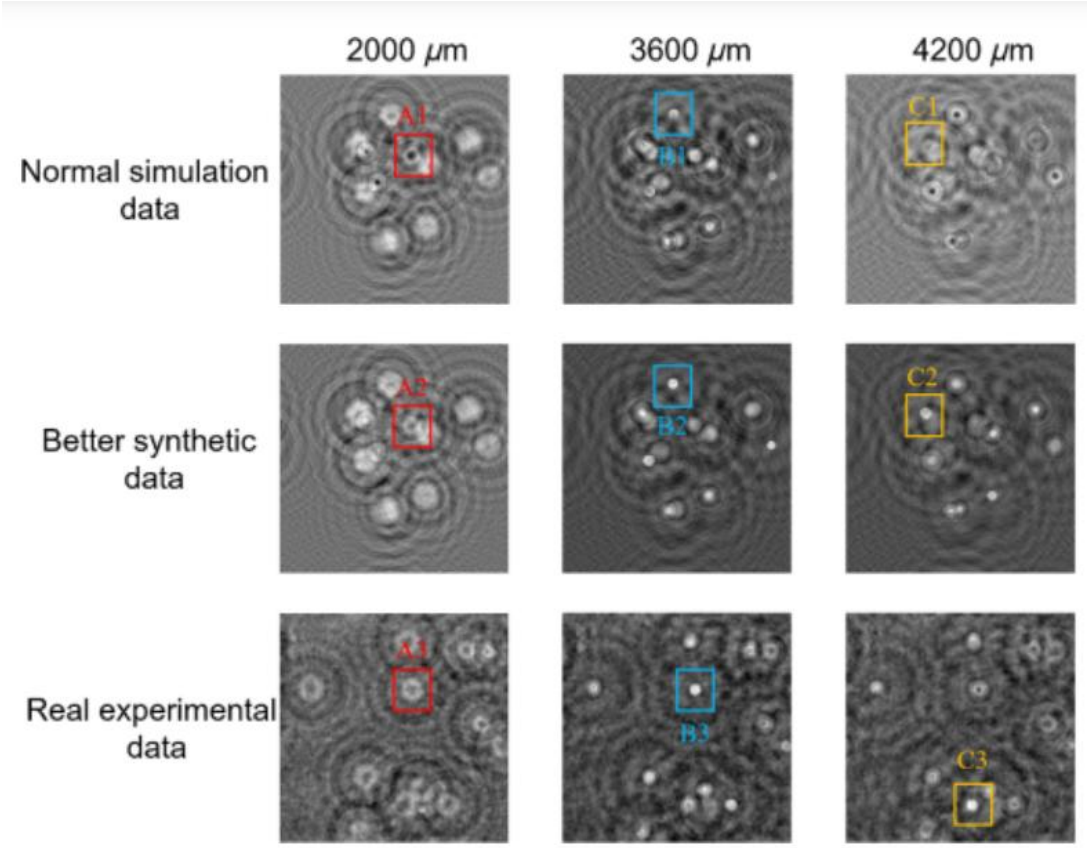
Data Augmentation

Data Augmentation no es lo mismo que Synthetic Data

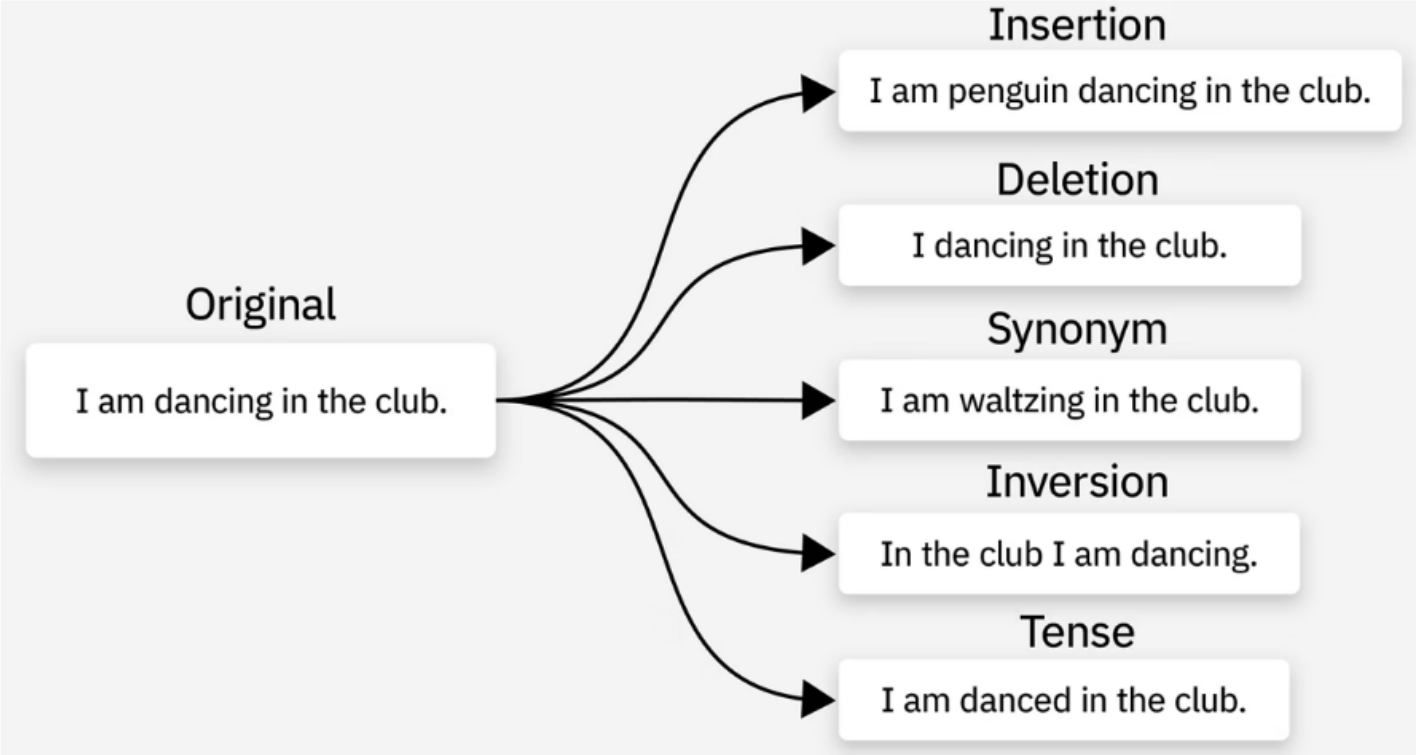
Consiste en **transformar datos reales existentes** para generar nuevas variaciones.



Consiste en **generar datos artificialmente** mediante modelos o simulaciones.



Técnicas de Data Augmentation



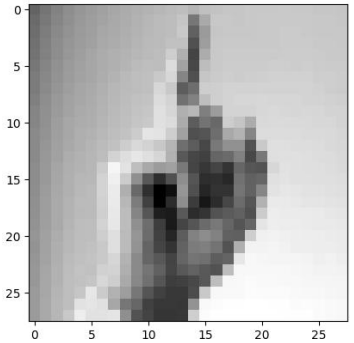
Link

Data Augmentation

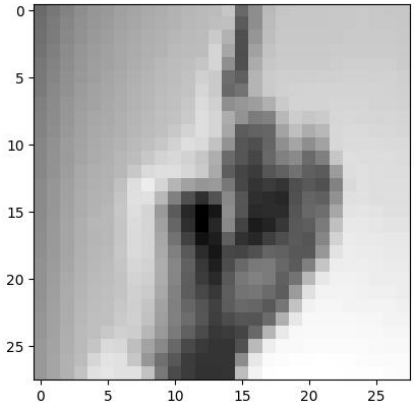
```
import torchvision.transforms.v2 as transforms
import torchvision.transforms.functional as F
```

```
row_0 = train_df.head(1)
y_0 = row_0.pop('label')
x_0 = row_0.values / 255
x_0 = x_0.reshape(IMG_CHS, IMG_WIDTH, IMG_HEIGHT)
x_0 = torch.tensor(x_0)
x_0.shape
```

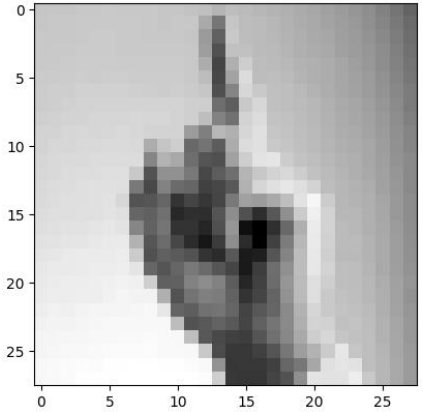
```
torch.Size([1, 28, 28])
```



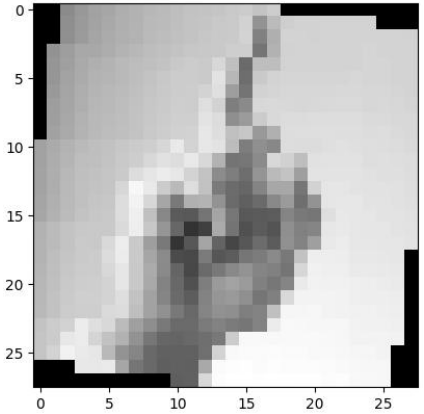
```
trans = transforms.Compose([
    transforms.RandomResizedCrop((IMG_WIDTH, IMG_HEIGHT), scale=(.7, 1), ratio=(1, 1)),
])
```

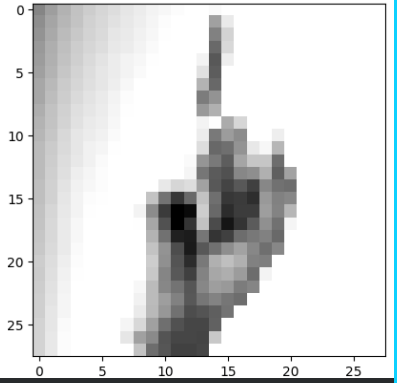


```
trans = transforms.Compose([
    transforms.RandomHorizontalFlip()
])
```



```
trans = transforms.Compose([
    transforms.RandomRotation(10)
])
```





```
brightness = .9  # Change to be from 0 to 1
contrast = .5  # Change to be from 0 to 1

trans = transforms.Compose([
    transforms.ColorJitter(brightness=brightness, contrast=contrast)
])
```

Desventajas y/o limitaciones

Las técnicas específicas utilizadas para aumentar los datos dependen de la naturaleza de los datos con los que trabaja el modelo.

Algunas transformaciones pueden generar imágenes que no tienen sentido en el contexto del problema.

Algunas transformaciones pueden cambiar el significado de la imagen.

Data augmentation no agrega nueva información real al dataset. Las transformaciones generan variaciones de los mismos datos, pero no nuevos ejemplos independientes.

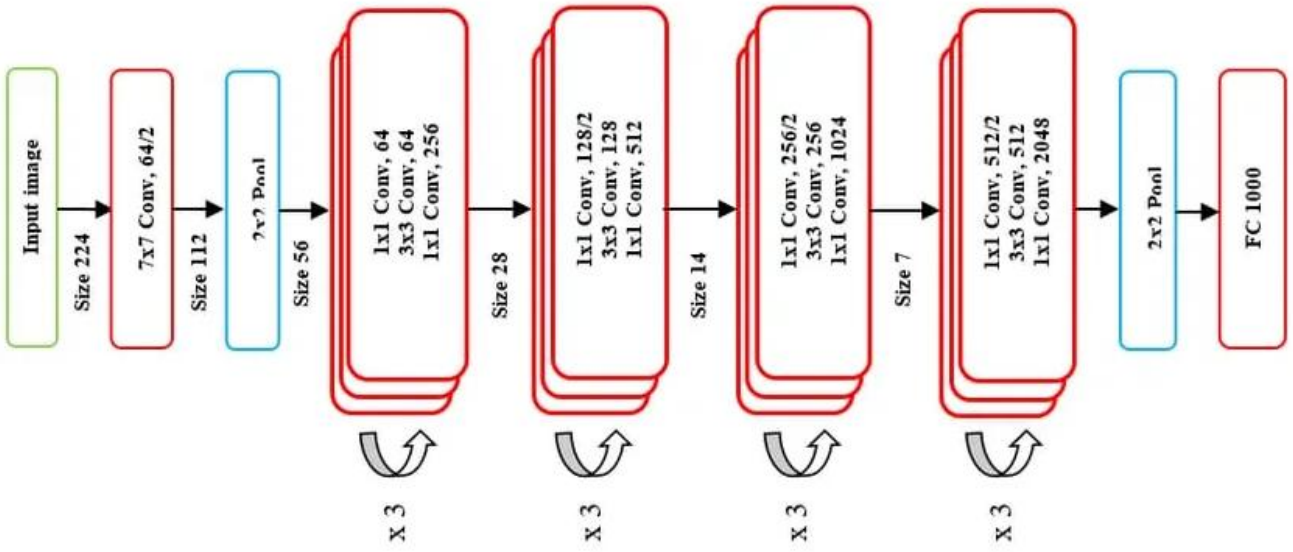
Si el dataset es muy pequeño o sesgado, el problema puede persistir.

Requiere conocimiento del dominio

Transfer Learning

Consiste en aprovechar el conocimiento **aprendido** en una tarea para resolver **otra tarea relacionada**.

Transfer Learning un modelo entrenado en un dataset **grande** se reutiliza para resolver un nuevo problema.

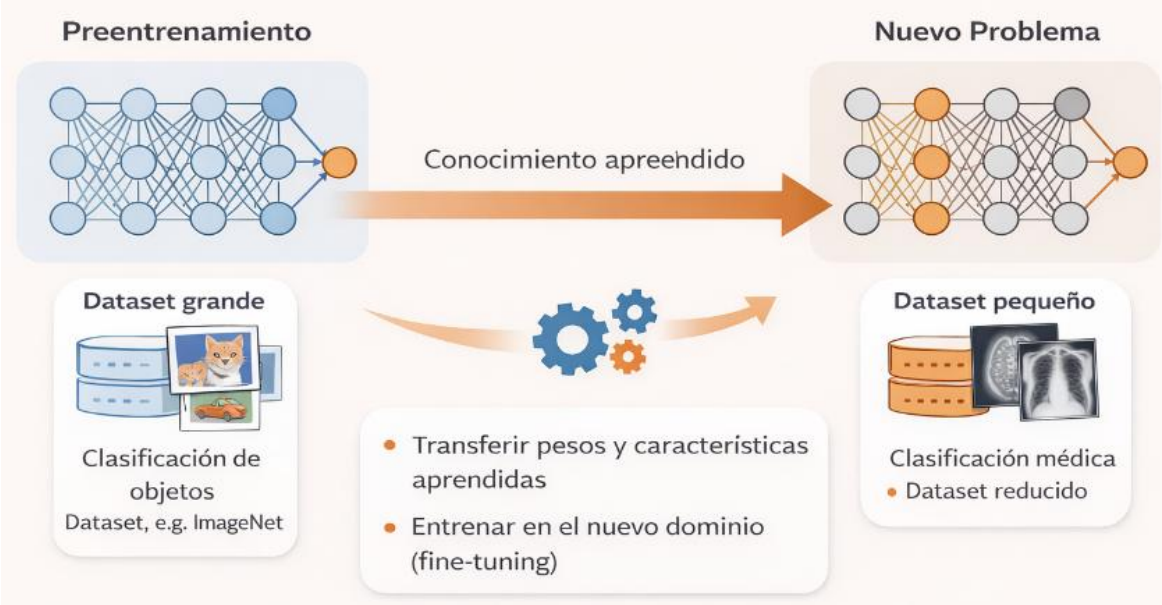


ResNet-50 architecture

* 1.2 millones de imágenes de entrenamiento * 1000 clases
* 50 000 imágenes de validación * 100 000 imágenes de test

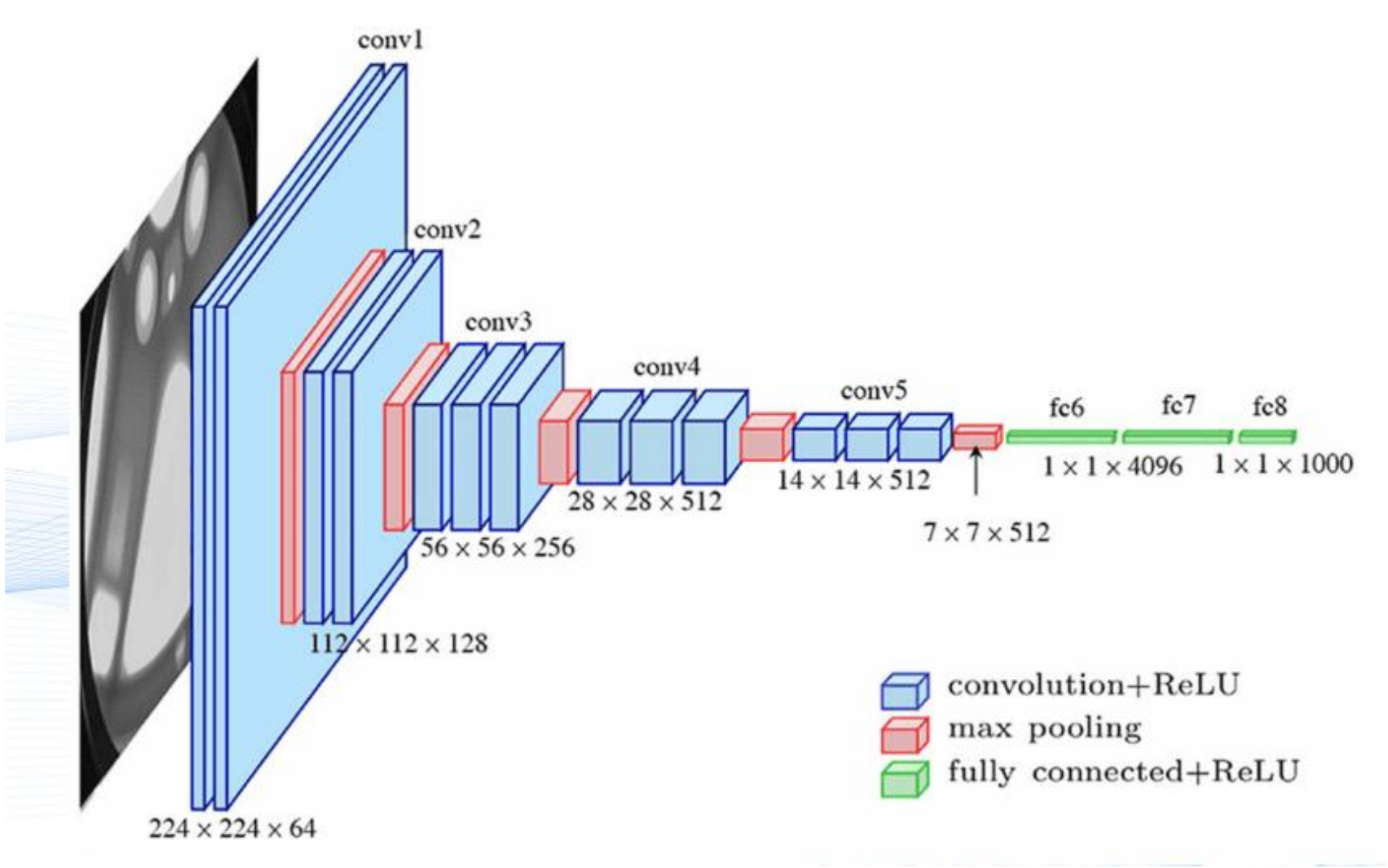
LIMITACIONES

- Grandes **datasets**
- **Tiempo de entrenamiento**
- **costo computacional**
- Los datos disponibles son **limitados**



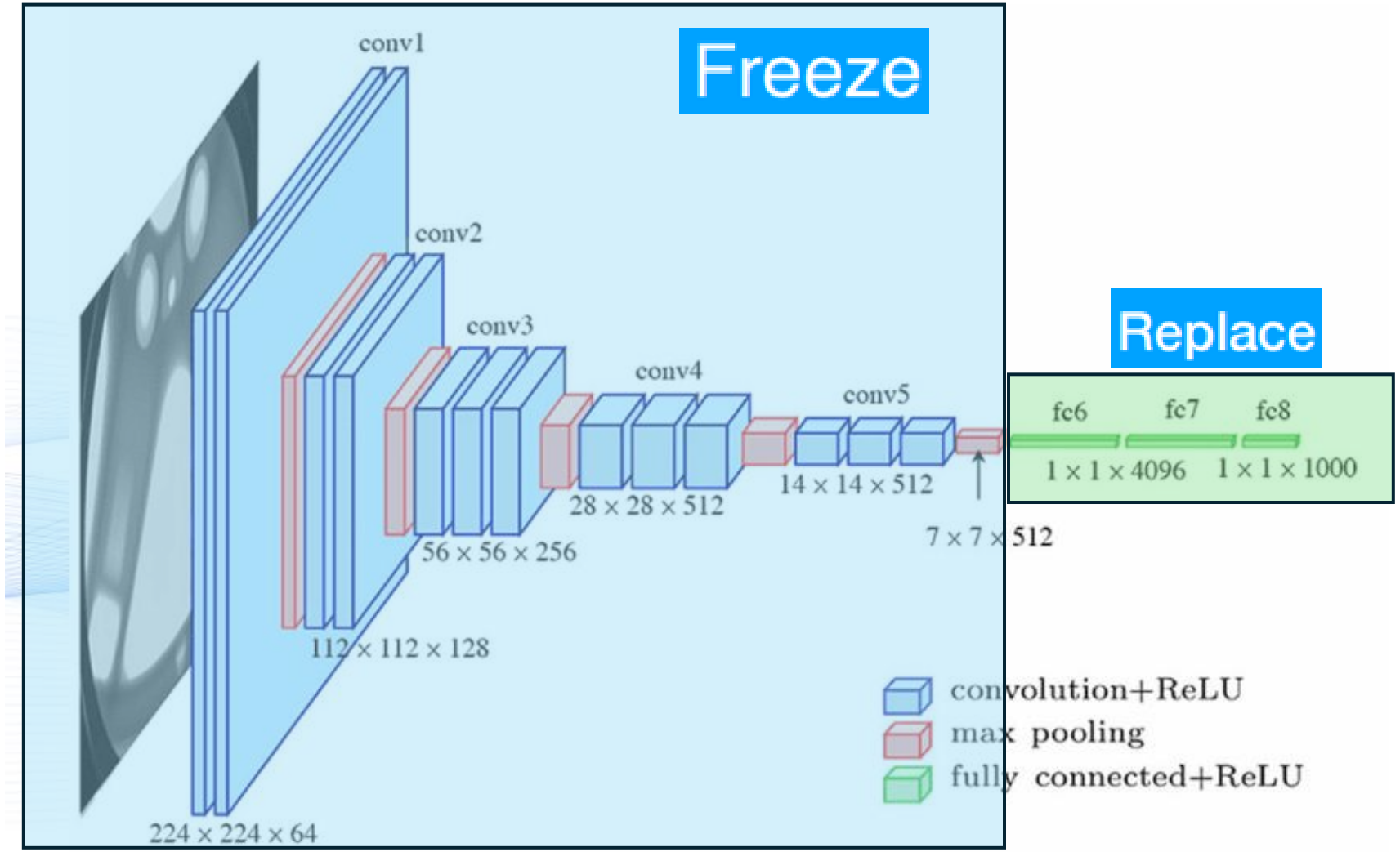
Kernels / Convolución

¿Por qué funciona?



Kernels / Convolución

¿Por qué funciona?



Las **primeras capas de una CNN** aprenden características generales: Bordes, texturas, patrones.

Las **últimas capas** aprenden características específicas de la tarea.

Transfer Learning

Estrategias de Transfer Learning

Feature Extraction

Se usa la red preentrenada como extractor de características.

- Las capas convolucionales se congelan
- Solo se entrena el clasificador final
- rápido
- requiere pocos datos

Fine-tuning parcial

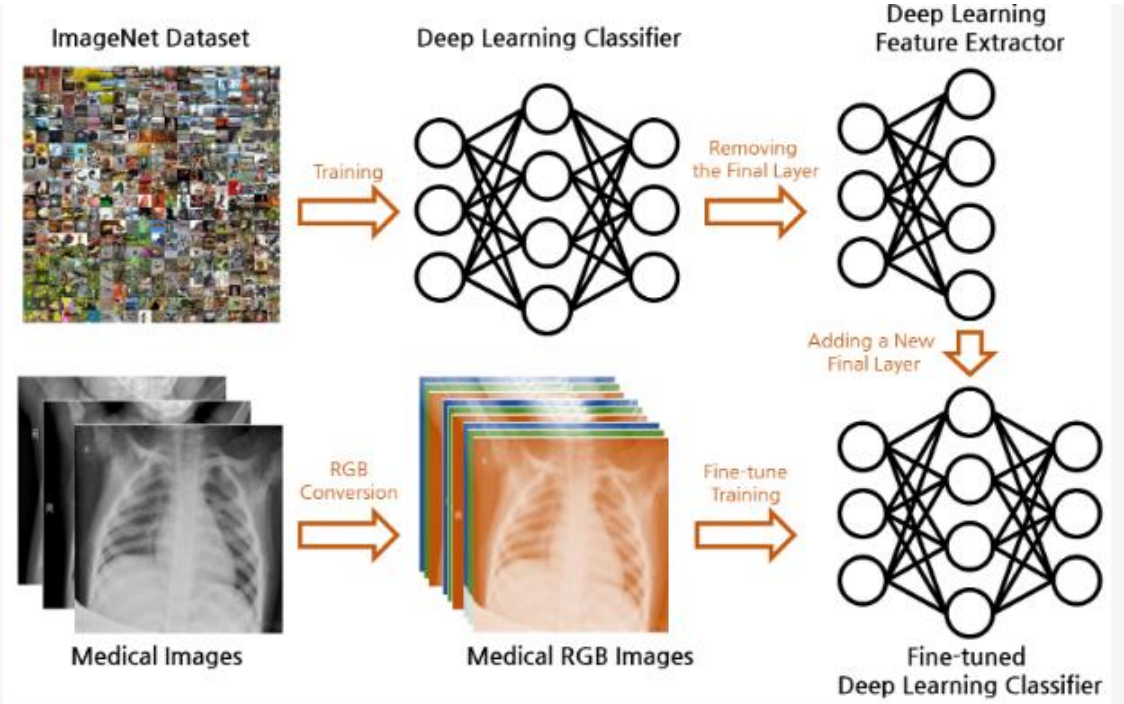
Se entrenan algunas capas superiores del modelo. Las capas iniciales permanecen congeladas.

- Mejor adaptación al nuevo problema
- Aprovechar el conocimiento previo

Complete-tuning

Se entrenan **todas las capas del modelo**.

- Usado si el dataset nuevo es grande
- Las tareas son similares
- Existe riesgo de Overfitting



Transfer Learning

¿Qué capas reemplazar y entrenar?

Large-scale Video Classification with Convolutional Neural Networks

Andrej Karpathy[1,2] George Toderici[1] Sanketh Shetty[1]
karpathy@cs.stanford.edu gtoderici@google.com sanketh@google.com
Thomas Leung[1] Rahul Sukthankar[1] Li Fei-Fei[2]
leungt@google.com sukthankar@google.com feifeili@cs.stanford.edu
[1]Google Research [2]Computer Science Department, Stanford University
http://cs.stanford.edu/people/karpathy/deepvideo

Link

Model	3-fold Accuracy
Soomro et al [22]	43.9%
Feature Histograms + Neural Net	59.0%
Train from scratch	41.3%
Fine-tune top layer	64.1%
Fine-tune top 3 layers	**65.4%**
Fine-tune all layers	62.2%

Table 3: Results on UCF-101 for various Transfer Learning approaches using the Slow Fusion network.

- Paper original del dataset
- Enfoque clásico de **computer vision**
- Se entrena la red **desde cero**
- **Transfer Learning -** Entrenar **solo la última capa.**
- **Transfer Learning -** Entrenar **las tres últimas capas.**
- **Transfer Learning -** Entrenar **toda la red preentrenada.**

UNIVERSIDAD EAFIT

Transfer Learning

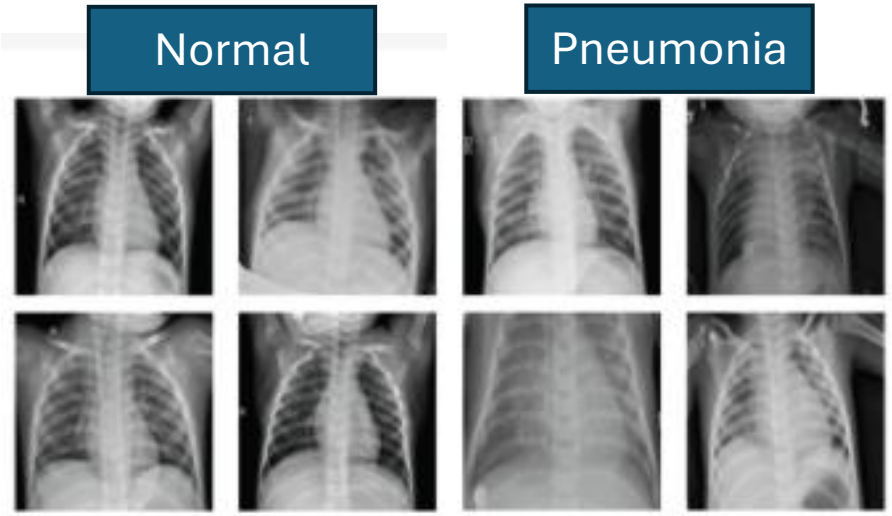
Table 3. Performance comparison of from-scratch training models vs. real-world feature transfer learning models with reduced learning rate. Bold text indicates the best results for each experimental setting.

	Model	Test Loss	Test Accuracy				
ResNet18	Scratch Model	0.288	89.1%	89.5%	93.3%	0.914	0.894
	Transfer Learning	**0.183**	**92.6%**	**90.8%**	**98.2%**	**0.944**	**0.901**
ResNet50	Scratch Model	0.395	82.4%	**88.1%**	83.2%	0.856	**0.855**
	Transfer Learning	**0.311**	**88.7%**	85.3%	**99.5%**	**0.919**	0.823
DenseNet161	Scratch Model	0.315	90.7%	88.8%	97.4%	0.929	0.881
	Transfer Learning	**0.176**	**92.9%**	**91.2%**	**98.4%**	**0.947**	**0.906**
ShuffleNet v2	Scratch Model	0.488	77.5%	79.2%	86.6%	0.827	0.782
	Transfer Learning	**0.372**	**90.4%**	**90.5%**	**94.9%**	**0.926**	**0.903**
MobileNet v2	Scratch Model	0.453	79.3%	**87.9%**	77.8%	0.825	0.842
	Transfer Learning	**0.261**	**90.1%**	87.6%	**98.4%**	**0.927**	**0.861**
ResNeXt50 (32x4d)	Scratch Model	0.441	80.4%	88.2%	79.3%	0.835	0.852
	Transfer Learning	**0.206**	**91.5%**	**90.3%**	**96.9%**	**0.935**	**0.901**
Wide ResNet50-2	Scratch Model	**0.305**	87.9%	**90.4%**	90.2%	0.903	**0.881**
	Transfer Learning	0.322	**89.0%**	85.6%	**99.5%**	**0.920**	0.830

Deep Transfer Learning Using Real-World Image Features for Medical Image Classification, with a Case Study on Pneumonia X-ray Images

by Chanhoe Gu [1] and Minhyeok Lee [1,2,*]

Link



Métricas de evaluación para clasificación

Table 4. Performance comparison of InceptionNet trained from scratch vs. real-world feature transfer learning.

Model	Test Loss	Test Accuracy	Precision	Recall	F1	G-Mean
Scratch Model	0.315	91.2%	89.7%	97.2%	0.933	0.892
Transfer Learning	**0.187**	**93.3%**	**91.4%**	**98.5%**	**0.948**	**0.907**